



SAKINAH

Implementation Plan

AS BUILT

Version 1.0 | 07 September 2026

Verified from the secured repository. Secret values excluded.

Contents

1. Current product status
2. Status inventory
 - Completed
 - Partially implemented
 - Missing or unverified
3. Delivery governance
4. Phase 0 - Security owner verification
5. Phase 1 - Quality baseline
6. Phase 2 - Toolchain security upgrade
7. Phase 3 - Data lifecycle and quota hardening
8. Phase 4 - Content completeness and licensing
9. Phase 5 - Reliability and performance
10. Phase 6 - Accessibility and UI consistency
11. Phase 7 - Notification architecture decision
12. Testing and documentation gaps
13. Deployment-readiness checklist
14. Traceable verification commands

Product: Sakinah | Status: Current-State Roadmap | Version: 1.0 | Date: 2026-09-07

1. Current product status

Sakinah is deployed as a functional mobile-first PWA with guest use, Supabase authentication/owner data, Quran/Hadith integrations, prayer/Qibla utilities, Tasbeeh, guidance, More tools, dark mode, install onboarding, offline fallback, and versioned updates. Security hardening is committed in 296fcac and documented in docs/security/SECURITY-AUDIT.md.

2. Status inventory

Completed

- Core PWA shell, install manifest, standalone onboarding, service worker/update flow.
- Bottom navigation, sidebar, responsive shell, light/dark mode.
- Supabase Auth client and RLS schema.
- Quran Surah/Juz reader, translation/audio preferences, favorites/bookmarks.
- Quran extras: Seerah structure, recitation, subjects, 38 Fahm lessons, line modes.
- Eight Hadith collection metadata and remote collection/topic workflow.
- Live prayer request/countdown and Qibla bearing/sensor integration.
- Preset/custom Tasbeeh and history.
- Duas, Ibadat, Flashes/Greetings, Shahadat, Names, Waqf.
- Security headers, validation, privacy cache fix, security tests.

Partially implemented

- Books: catalog exists; full licensed downloadable books are not verified.
- Hadith translations: availability depends on provider editions.
- Seerah: seven-season UI indexes a fixed external playlist.
- Notifications: browser-local scheduling, not reliable server push.
- Profile: basic user/saved summary rather than full preference management.
- Offline: shell/images/selected Quran cache, not complete corpus download.

Missing or unverified

- Admin/CMS, account deletion/export, server push scheduler, full book storage, analytics/observability, remote Supabase security evidence, end-to-end browser tests, lint/type-check, accessibility audit.

3. Delivery governance

1. Work one phase at a time.
2. Do not start the next phase until tests pass and the phase is committed.
3. Update PRD/TRD before implementing changed requirements.
4. Preserve working code outside the phase.
5. Obtain owner confirmation for content licensing, provider substitution, schema semantics, auth behavior, or architecture changes.

4. Phase 0 - Security owner verification

Goal: Close controls that cannot be proved from source.

Current status: Required owner action.

Tasks

- Verify remote RLS with two dedicated test accounts.
- Confirm Supabase Site URL/redirect allow-list, Google callback, CAPTCHA, auth rate limits, token lifetime/reuse protection, and SMTP settings.
- Confirm Vercel variables contain only public browser values.
- Check deployed CSP without weakening it.

Dependencies: Owner access to Supabase, Google Cloud, Vercel; non-production test users.

Likely files: Documentation only unless a verified mismatch requires an approved patch.

Risks: Production auth interruption from incorrect redirects/policies.

Tests: Cross-owner negative read/write; signup/login/reset/OAuth; security-header/CSP browser test.

Done when: Evidence is attached to an owner-controlled audit record and all negative authorization tests pass.

Do not build/change yet: Database policies or Auth settings without backups and exact current-state capture.

5. Phase 1 - Quality baseline

Goal: Add trustworthy regression coverage without redesign.

Status: Planned.

Tasks: Select one existing-compatible browser test tool; cover guest launch, navigation, auth mock boundaries, Quran provider failure, prayer permission denial, Tasbeeh validation, PWA manifest; add lint/type-check only after owner approves conventions.

Dependencies: Phase 0; test accounts/API fixtures.

Files: `tests/`, `package.json`, possible non-production test config.

Risks: Brittle live-provider tests. Use fixtures/mocks for CI.

Verification: `npm run test:security`, future E2E command, `npm run build`, `npm audit --omit=dev`.

Done when: Critical navigation and owner-data paths have deterministic positive/negative tests.

Do not build/change yet: Product features or UI redesign.

6. Phase 2 - Toolchain security upgrade

Goal: Remove Vite/esbuild development advisories.

Status: Planned; owner approval required for major versions.

Tasks: Pin supported Node version; create upgrade branch; update Vite/React plugin compatibly; compare output, PWA, CSS, OAuth callback, and local mobile workflow; regenerate lockfile only as part of the approved upgrade.

Dependencies: Phase 1 coverage.

Files: `package.json`, `package-lock.json`, `vite.config.js`, CI/runtime config.

Risks: Node engine incompatibility and build behavior changes.

Done when: Full suite/build pass and `npm audit` has no applicable advisory.

Do not build/change yet: React major, Supabase provider, or hosting architecture.

7. Phase 3 - Data lifecycle and quota hardening

Goal: Bound user-owned storage and provide privacy lifecycle.

Status: Proposed, not approved.

Tasks: Measure existing payloads; design non-breaking DB constraints/indexes; define account deletion/export requirements; decide whether custom images move to private Supabase Storage; add migrations and rollback plan.

Dependencies: Owner product decisions, database backup, Phase 0 evidence.

Files: future migrations, data services, PRD/TRD first.

Risks: Existing oversized rows, data loss, storage cost.

Tests: migration against sanitized copy; quota boundaries; cascade/delete/export; RLS.

Done when: Approved retention policy and migration pass without production data loss.

Do not build/change yet: Do not alter `schema.sql` or production data before assessment.

8. Phase 4 - Content completeness and licensing

Goal: Establish a verified, licensed corpus contract.

Status: Partially implemented/unverified.

Tasks: Inventory each Quran translation, reciter, Hadith edition, Dua, book, image, font, and Seerah source; record license/attribution; define completeness checks; replace broken content only with owner/content-review approval.

Dependencies: Qualified Islamic content reviewer and licensing review.

Files: content adapters/assets and documentation after approval.

Risks: Incorrect religious text, broken attribution, copyright issues.

Tests: counts/checksums/provider schema, Arabic/Urdu glyph review, broken-link test.

Done when: Each corpus has provenance, license, expected count, and reviewer sign-off.

Do not build/change yet: Do not describe partial book/translation sets as complete.

9. Phase 5 - Reliability and performance

Goal: Improve measured load behavior and offline consistency.

Status: Existing foundations; further work planned.

Tasks: Capture Web Vitals on target Android/iOS; split the large App bundle further; virtualize large Hadith/Quran lists; add cache TTL/size policy; test update lifecycle and offline fallback; preserve coordinate privacy.

Dependencies: Phase 1 tests and content-provider contract.

Files: component boundaries, service worker, Vite config.

Risks: stale data, broken lazy routes, platform-specific PWA behavior.

Tests: cold/warm load, five reloads, offline/online transition, installed update, storage quota.

Done when: Owner-approved performance budgets pass on representative devices.

Do not build/change yet: Do not add aggressive caching for Auth, Supabase, or coordinate URLs.

10. Phase 6 - Accessibility and UI consistency

Goal: Make the existing design system more usable without rebranding.

Status: Planned.

Tasks: Standardize focus-visible states; audit contrast and 8–10 px labels; import/verify Arabic font; review no-selection policy; complete labels/roles; extend reduced-motion handling; test RTL content.

Dependencies: Accessibility acceptance criteria in PRD.

Files: CSS and components only after doc approval.

Risks: subtle layout changes; scripture typography regression.

Tests: keyboard-only, screen reader, zoom 200%, contrast, reduced motion, RTL screenshots.

Done when: Agreed WCAG target passes and branding remains intact.

Do not build/change yet: No palette/logo/navigation replacement.

11. Phase 7 - Notification architecture decision

Goal: Align reminder promises with browser platform limits.

Status: Decision required.

Tasks: Document current local reminder limitations; decide between local-only and true Web Push; if push is approved, define consent, subscription storage, scheduler, VAPID secret handling, timezone, unsubscribe, and privacy requirements before implementation.

Dependencies: Owner scope, privacy policy, backend architecture decision.

Risks: notification spam, secret handling, battery/platform restrictions.

Done when: Product wording matches tested delivery guarantees.

Do not build/change yet: Do not claim automatic Azan/background reliability without device evidence.

12. Testing and documentation gaps

- No automated browser E2E or component test suite.
- No lint/type-check command.
- No remote RLS test artifacts.
- No API schema contract tests or provider fixtures.
- No documented recovery/runbook or privacy policy.
- No formal content provenance register.

13. Deployment-readiness checklist

- Security owner verification complete.
- Tests/build/audit pass from clean checkout.
- Vercel headers/CSP verified.
- Supabase variables and redirects reviewed.
- PWA install/update tested on actual Android and iPhone.
- Provider failure/offline states tested.
- Content/license register approved.
- Rollback commit/deployment identified.

14. Traceable verification commands

```
npm install
npm run test:security
npm run build
npm audit --omit=dev
git diff --check
git status --short --branch
```

Every implementation phase must list its changed files, test output, unresolved risks, and commit before the next begins.